Visualizing Sensors Data with RViz2 (Cameras, LIDARs, Pose Estimation, Distance Measurement)

Introduction to Robotics Lab

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

Department of Electrical and Computer Engineering
University of Patras

November 25, 2025

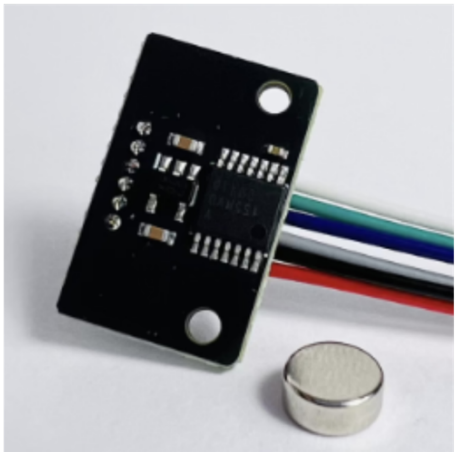
Table of Contents

1. Sensors Introduction
   - Connection with RViz2 Lab
     - Encoders
   - Sensor Categories
   - Well known Sensors
2. Cameras
   - Key Uses in Robotics
   - Pinhole Camera Model
   - Pinhole Model Maths
   - Camera Parameters
   - Camera Calibration
3. LIDARs
   - What is a LIDAR? ▪ Key Uses in Robotics
   - Basic Working Principle
   - LIDAR Parameters
4. Camera Exercise ▪ ArUco Detection & Pose Estimation
   - Logitech C270 HD Webcam
   - ArUco markers
   - v4l2_camera & OpenCV
   - Part 1: `Image` msg & `TransformStamped` msg
   - Part 2: ArUco Setup
   - Part 3: RViz2 Visualize
5. LIDAR Exercise ▪ Distance Measurement & Radar Setup
   - RPLIDAR A1M8
   - rplidar_ros
   - Part 1: `LaserScan` msg
   - Part 2: Radar Setup
   - Part 3: RViz2 Visualize
6. References

Sensors Introduction - Connection with RViz2 Lab ▪ Encoders

We have already used sensors in the lab! Both the Elephant MyArm 300 Pi robotic arm and the MAB double pendulums rely on **absolute magnetic encoders** to ***continuously measure joint positions***. Absolute encoders retain their angle after power-up, enabling ***immediate and accurate RViz2 joint visualization***.

1. **MyArm 300 Pi** uses **7 magnetic encoder motors**. Based on observed values (e.g., zero at 2048), the encoders appear to operate with **12-bit resolution (4096 counts)**, implying a precision of approximately 0.1° per count.
2. **Double pendulum (MAB Robotics)** uses **two MD80 motor drivers with ME-series (Magnetic Encoder) modules**. These contain **14-bit absolute magnetic encoders** offering about 0.1° accuracy on-axis and around 1° in off-axis configurations.



On-axis Encoder



Off-axis Encoder

Sensors Introduction - Sensor Categories

1. **Analog vs. Digital Sensors**
   - Analog: Provide continuous signals (e.g., *thermistor*, *strain gauge*)
   - Digital: Provide discrete signals or digital data (e.g., *digital temperature sensor*, *IMU*)
2. **Active vs. Passive Sensors**
   - Active: Require external energy to operate (e.g., *radar*, *ultrasonic*)
   - Passive: Work by receiving or converting existing energy (e.g., *thermocouple*, *photodiode*)
3. **Proprioceptive vs. Exteroceptive Sensors**
   - Proprioceptive: Sense the robot's own physical state (e.g., *joint encoders*, *wheel odometry*, *gyroscope*, *accelerometer*, *IMU*)
   - Exteroceptive: Measure the external environment state (e.g., ***camera***, ***LIDAR***)

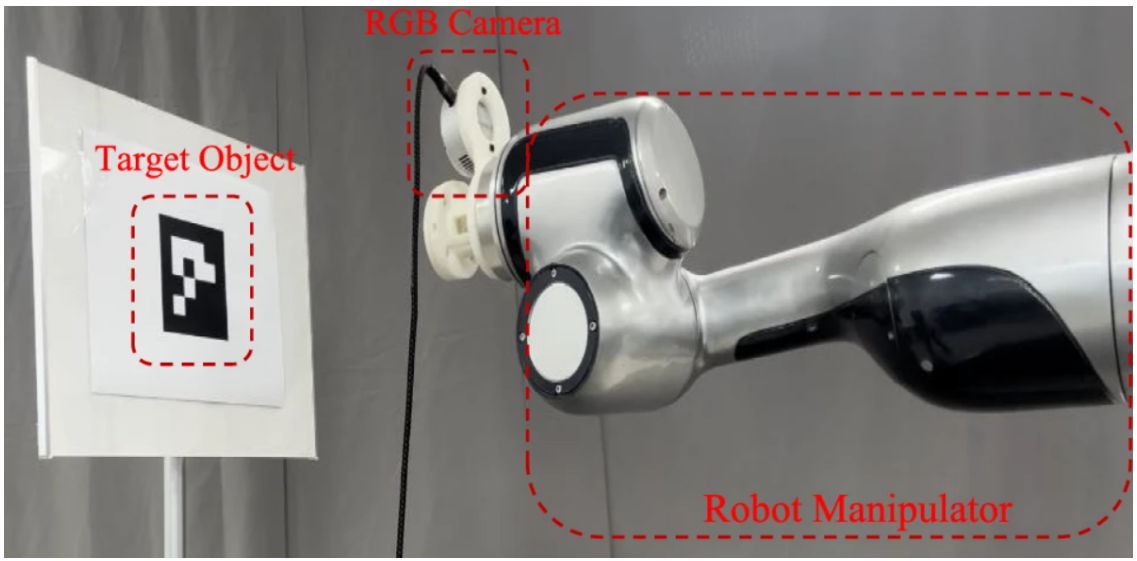
Sensors Introduction - Well known Sensors

1. **Potentiometers** → position/angle sensing
2. **Load Cells** → strain-gauge based force measurement
3. **Rotary & Linear Encoders** → absolute or incremental
4. **Cameras** → RGB, IR, depth
5. **LIDARs** → 2D/3D scanning
6. **IMUs** → accelerometer + gyroscope + magnetometer
7. **Thermistors** → temperature sensing
8. **Microphones** → acoustic sensing
9. **GPS** → global position sensing
10. **Proximity Sensors** → infrared, capacitive, inductive, ultrasonic
11. **Pressure Sensors** → barometric, piezoresistive

Cameras - Key Uses in Robotics

We all know what a camera is! In Robotics, it is used as an exteroceptive sensor that ***captures light to produce 2D images or video***, allowing robots to ***perceive and interpret their environment***. Common applications in Robotics are:

1. **Object recognition** and **scene understanding**
2. **Navigation** and **SLAM** (Simultaneous Localization and Mapping)
3. **Motion planning** and visual **feedback control**
4. *Manipulation tasks*: **grasp detection**, **pose estimation**, and **visual servoing**
5. *Human–robot interaction*: **gestures recognition** and **safety monitoring**
6. *Real-time tracking*: **autonomous driving**
7. *Industrial applications*: **assembly guidance** and **equipment inspection**

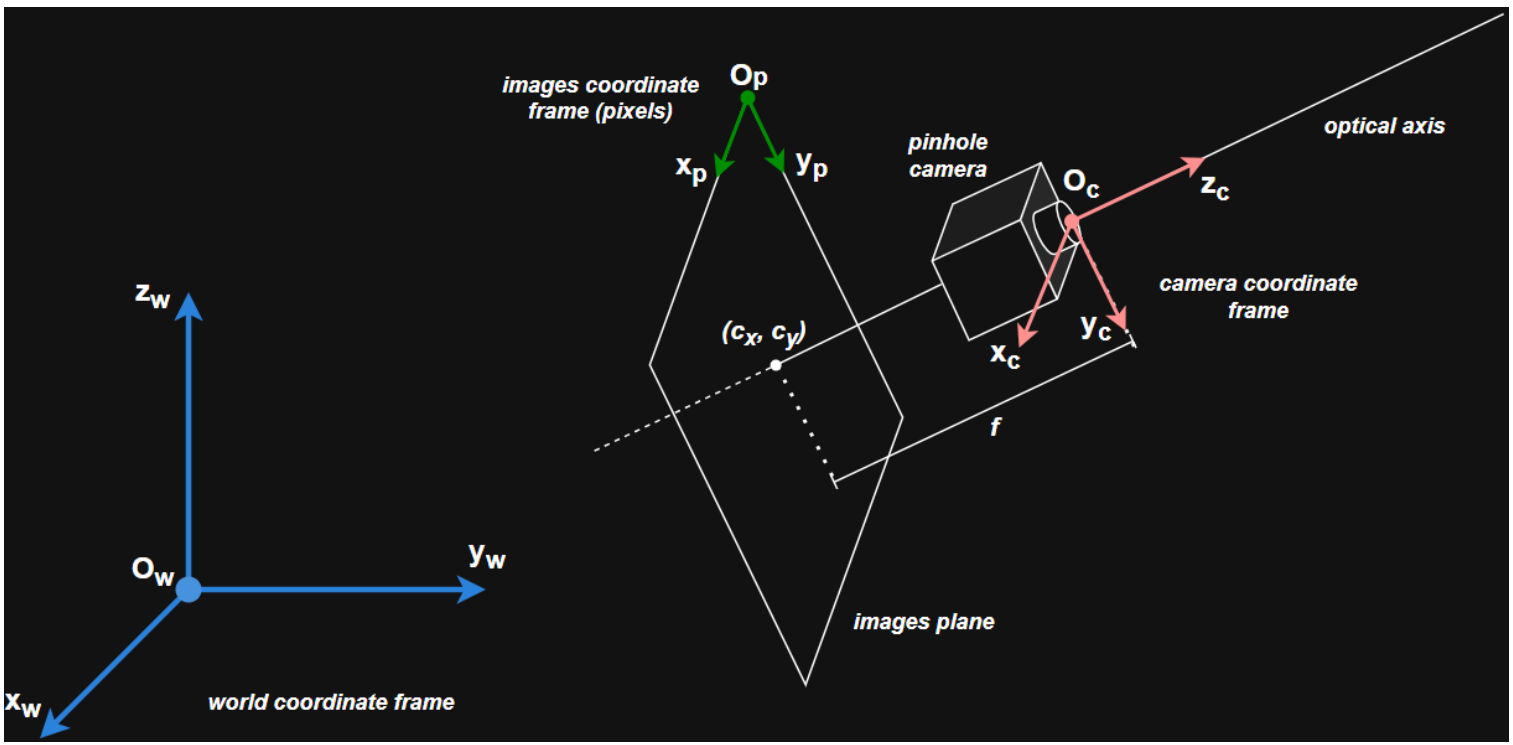


Position-based Visual Servoing example: An eye-in-hand mounted camera detects an ArUco marker (the target object), facilitating pose estimation and enabling robotic servo control

Cameras - Pinhole Camera Model

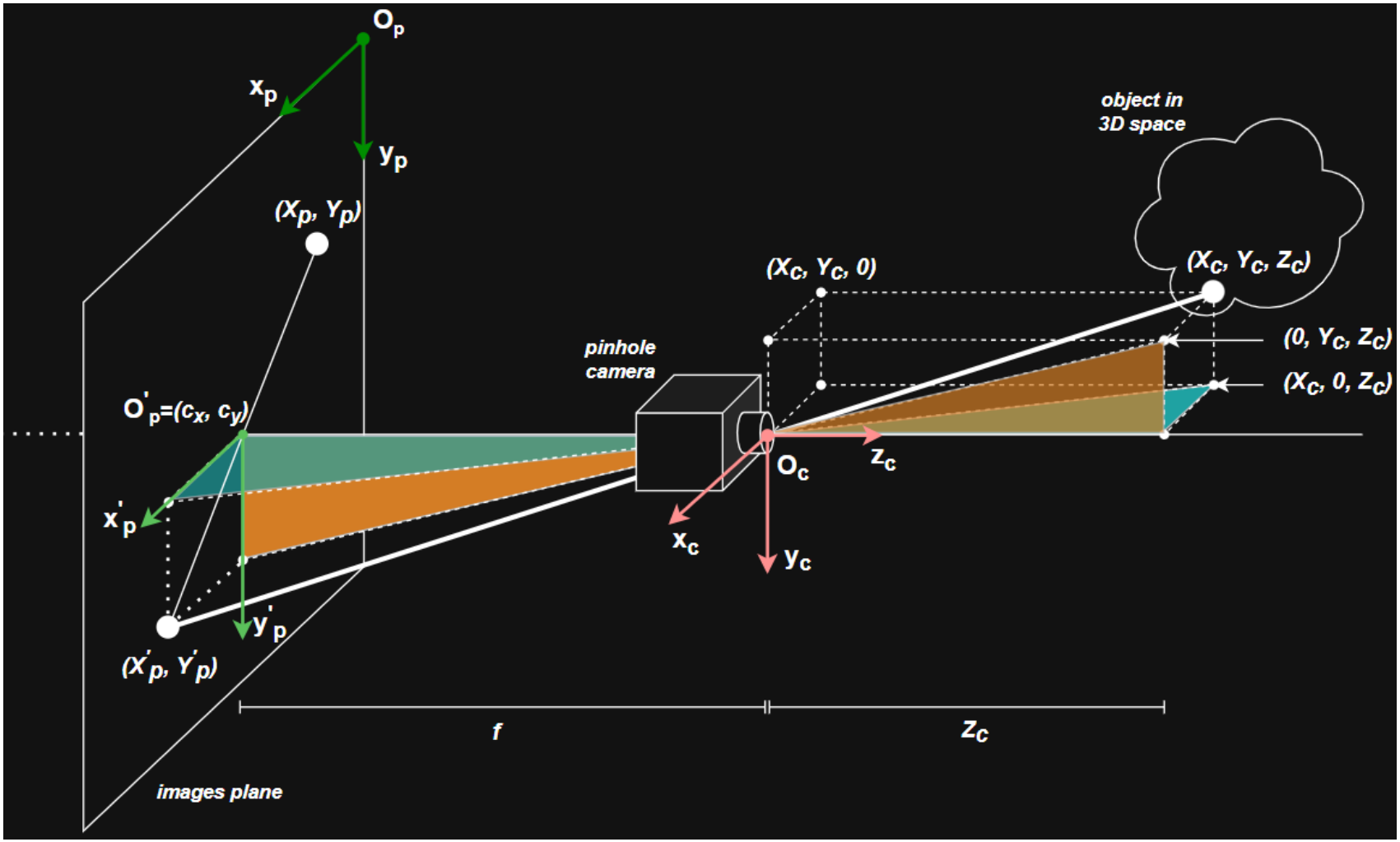
The **linear pinhole camera model** describes a transformation from the points $[X_w, Y_w, Z_w]^\top$ in 3D space to the pixels $[X_p, Y_p]^\top$ in 2D camera frames, on the image plane. The basic scheme is:

World Frame 3D Points (in m) $\xrightarrow{\text{extrinsic tf}}$ Camera Frame 3D Points (in m) $\xrightarrow{\text{intrinsic tf}}$ Image Plane 2D Points (in pixels)



Pinhole Camera Model

Cameras - Pinhole Camera Model



Projection from camera frame onto the image plane, where f is the focal length (in m), and (c_x, c_y) is the principal point (in pixels)

Cameras - Pinhole Model Maths

1. **Linear transformations**:

$$\begin{bmatrix} X_p \\ Y_p \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Intrinsic Matrix}} \cdot \begin{bmatrix} X_c/Z_c \\ Y_c/Z_c \\ 1 \end{bmatrix}, \quad \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} (R_c^w)^{-1} & -(R_c^w)^{-1} \cdot t_c^w \\ 0_{1\times 3} & 1 \end{bmatrix}}_{\text{Extrinsic Matrix}} \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

where R_c^w, t_c^w are the rotation and translation respectively from world to camera

2. **Image Plane Projection**:

$$X_p = -s_x \cdot X_p' + c_x \overset{X_p' = -f \cdot \frac{X_c}{Z_c}}{=} (f \cdot s_x) \cdot \frac{X_c}{Z_c} + c_x \overset{f_x = f \cdot s_x}{=} f_x \cdot \frac{X_c}{Z_c} + c_x \quad \text{(similarly for y axis)}$$

where f is the focal length (mm), and s_x, s_y are pixel scaling factors (pixels/mm)

(Note: For simplicity, distortion introduced by the camera lens during projection is ignored. It can later be corrected on the image plane using polynomial warping with the distortion coefficients. For full accuracy, distortion should be included in the projection model analysis. We also assume orthogonal image axes, since skew is typically negligible.)

Cameras - Camera Parameters

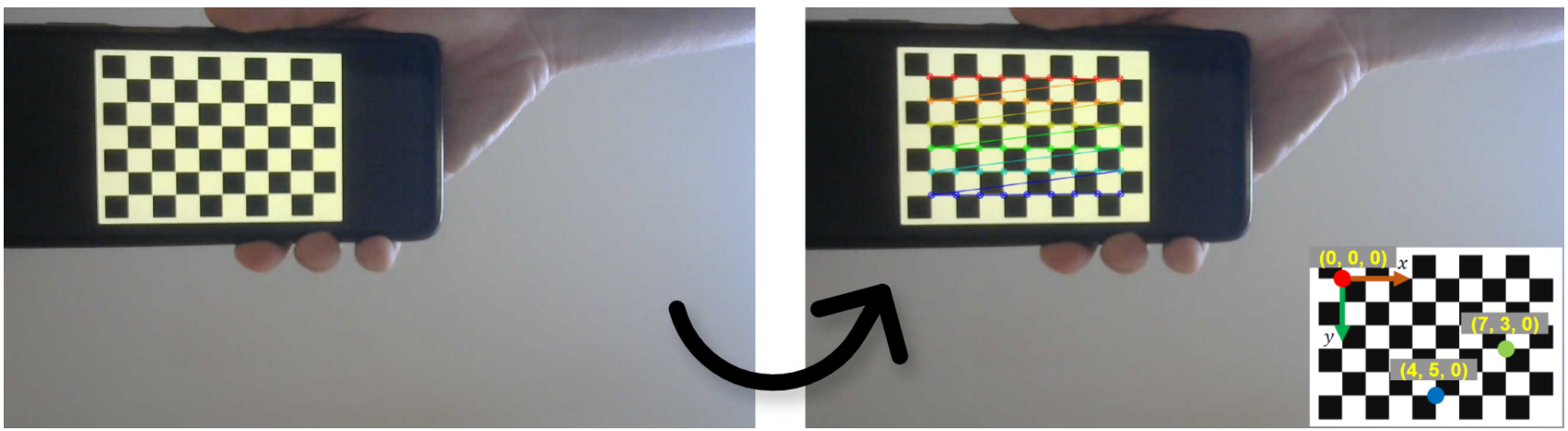
1. *Camera - specific*
   - **Resolution**: pixel count
   - **Aspect ratio**: width-to-height
   - **Field of View**: angle coverage
   - **Frame rate**: images per second
2. *Extrinsic*
   - **Position**: camera location
   - **Orientation**: camera rotation
3. *Intrinsic*
   - **Focal length**: lens magnification
   - **Principal point**: optical center
   - **Pixel scaling**: pixel size factor
   - **Skew**: axis non-orthogonality
4. *Lens Distortion*
   - **Radial**: barrel/pincushion warping
   - **Tangential**: lens misalignment shift



Logitech C270 HD Webcam

Cameras - Camera Calibration

Camera Calibration using a ***chessboard*** allows us to determine the ***internal parameters*** of a camera and correct ***lens distortion***. The process starts by taking multiple photos of a chessboard picture from different positions and angles, covering most of the camera's field of view. Next, a special software (e.g. OpenCV) detects the corners of the squares in each image with high precision and can refine their positions for more accuracy. These ***detected 2D points are then matched with the known 3D coordinates of the chessboard corners in the real world***. Finally, a calibration function estimates the camera's internal matrix and distortion coefficients by ***minimizing the difference between the projected 3D points and the detected 2D points***. The result is an ***accurate model of how the camera maps the 3D world onto the 2D image***.

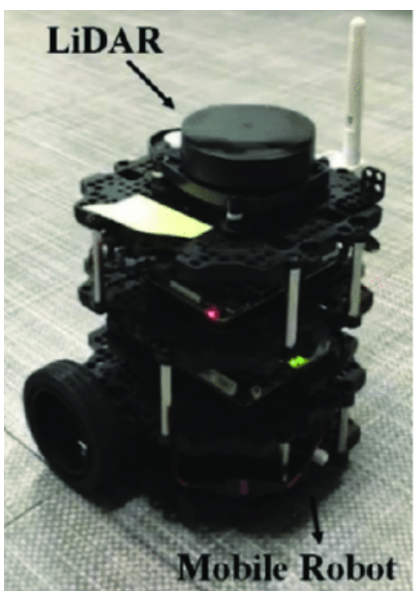


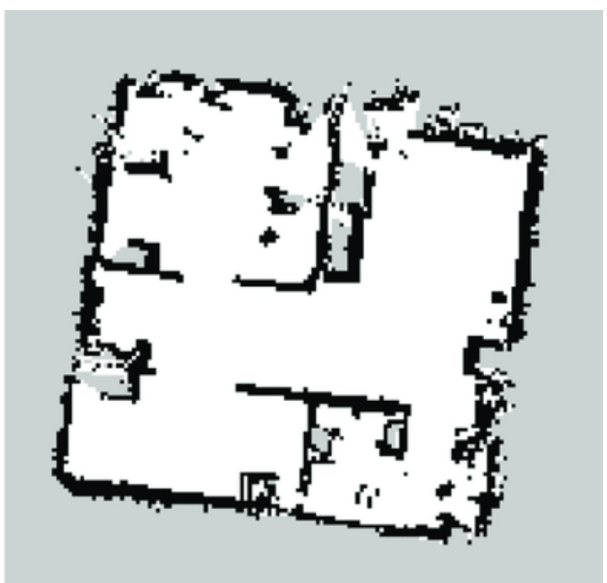
Camera Calibration process, image sample

LIDARs - What is a LIDAR? ▪ Key Uses in Robotics

LIDAR (short for **LIght** -or Laser Imaging- **Detection And Ranging**) is a remote-sensing technology for ***determining ranges by targeting an object or a surface with laser beams*** and ***measuring the time for the reflected light to return to the receiver***. It may operate in a fixed direction or may scan multiple ones, in a special combination of ***3D scanning and laser scanning***. Usual applications in Robotics are:

1. **Mapping and SLAM**: *generation of 3D maps* in indoor and outdoor environments
2. **Collision avoidance**: to *identify nearby objects* and safe paths for mobile robots
3. **Autonomous navigation**: *detection of road edges, obstacles, and moving objects*
4. **Environmental analysis**: *generation of detailed topographic maps* (measuring terrain, vegetation, and structures) to support robotic decision-making





Occupancy grid map built by a mobile robot with a LIDAR sensor

LIDARs - Basic Working Principle

LiDAR operates on the same fundamental idea as *radar* and *sonar*: *emit energy and analyze the reflected signal to detect objects*. The difference lies in the type of energy wave used; radar uses radio waves, sonar uses sound, while ***LiDAR relies on laser light***. A laser scanner and its associated electronics (including *timing circuits*, *processors*, and often *IMU* and *GPS* modules for precise positioning) ***capture the reflections*** and ***calculate distances*** with high accuracy, enabling detailed ***3D environmental mapping***.

1. A LIDAR **emits rapid pulses** of laser light toward a target.
2. The **laser pulses reflect** off surfaces and return to the LIDAR sensor.
3. The system measures the **"time of flight"** of each pulse.
4. Using the speed of light, the **distance to each point** is calculated.
5. Repeating this across many directions and points produces a 3D **point cloud**.

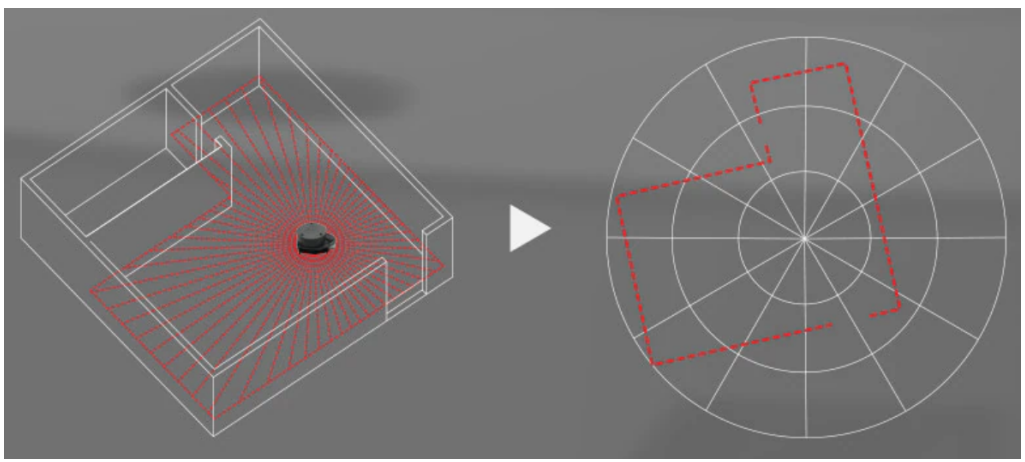


Diagram of a LIDAR's laser scanning process for generating a room's map

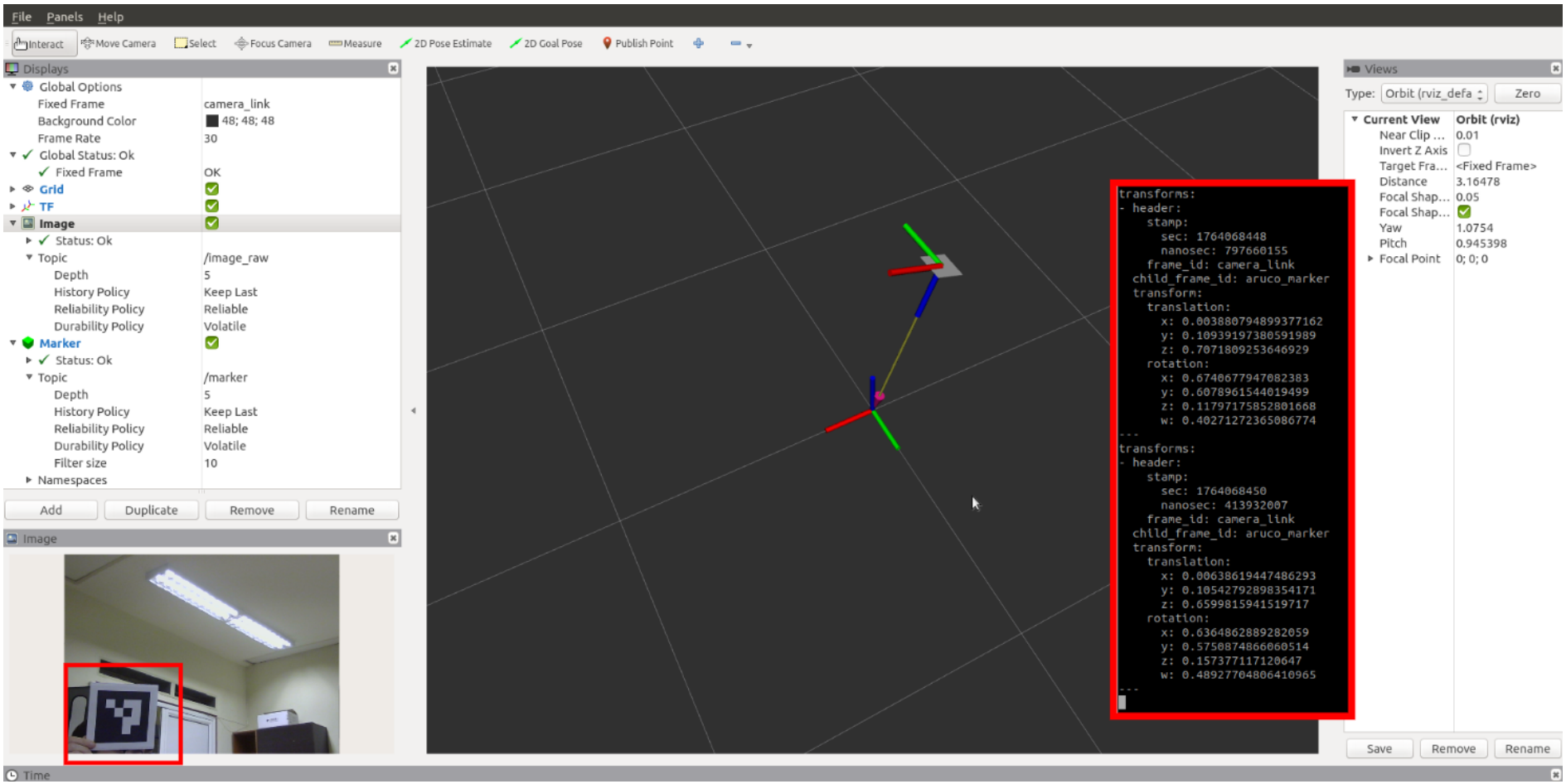
LIDARs - LIDAR Parameters

1. *LIDAR - specific*
   - **Detection Range**: maximum reliable measurement distance
   - **Distance Resolution**: smallest detectable change in distance
   - **Angular Range / Scanning Coverage**: total angular Field of View
   - **Angular Resolution**: minimum angle between consecutive measurements
   - **Scan Rate / Rotational Speed**: sensor rotations per second (in Hz)
   - **Sampling/Data Rate**: number of distance samples recorded per second
   - **Laser Wavelength**: wavelength of emitted laser (in nm)
2. *Extrinsic*
   - **Position**: LIDAR location
   - **Orientation**: LIDAR rotation



RPLIDAR A1M8

Camera Exercise ▪ ArUco Detection & Pose Estimation

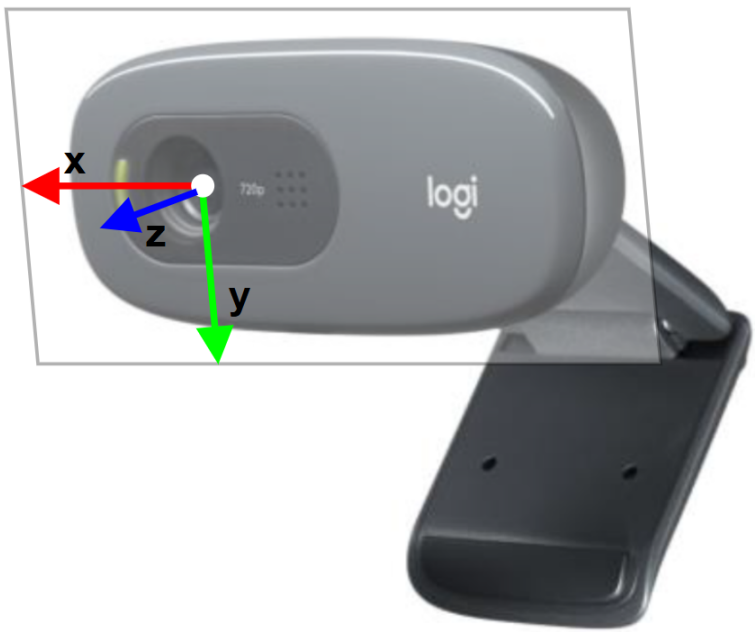


ArUco Detection and Pose Estimation - RViz2 view

Camera Exercise ▪ ArUco Detection & Pose Estimation - Logitech C270 HD Webcam

We're going to use the **Logitech C270 HD Webcam**. Some of its specifics are:

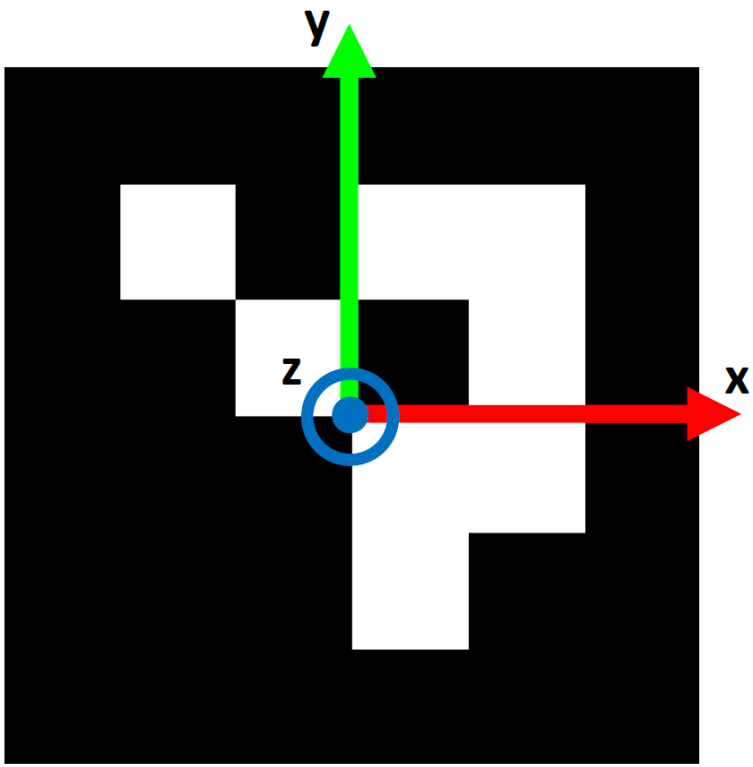
1. **720p HD resolution** (1280x720, *0.9 MP*)
2. Supports **16:9 and 4:3 aspect ratios**
3. **Diagonal FoV of** 55°
4. **Fixed focus** (also *fixed focal length*) with **automatic exposure adjustment**
5. **Up to 30 fps** capture rate
6. **RGB camera** (*no depth sensing*; distance must be inferred via references or algorithms)
7. **Minor lens distortion**



Logitech C270 HD Webcam, along with its coordinate frame

Camera Exercise ▪ ArUco Detection & Pose Estimation - ArUco markers

1. **ArUco markers** (Augmented Reality University of Cordoba) were developed at the University of Cordoba in Spain.
2. They are **square binary grids** composed of white and black tiles. The arrangement of the tiles encodes a **unique ID** for each marker.
3. The outer border is always black, while the inner region varies in the number of squares and patterns. ArUco markers are grouped into **predefined dictionaries**, each containing markers with different *sizes* and *encoding schemes*.
4. They are commonly used for ***camera pose estimation***, ***calibration***, ***object tracking***, ***augmented reality applications***, and ***robotics navigation***.



ArUco marker 4x4 (id=0), along with its coordinate frame

Camera Exercise ▪ ArUco Detection & Pose Estimation - v4l2_camera & OpenCV

We are going to use the ROS2 package **v4l2_camera** for capturing camera feed in Linux, and **OpenCV** software for image processing.

1. **v4l2_camera (Video4Linux2)**
   - Standard Linux interface for camera capture
   - ROS2 driver providing camera images as `sensor_msgs/Image` messages in topics like `/image_raw`
   - Configurable parameters, such as resolution, fps, image format, etc.
2. **OpenCV** (**Open** source **Computer Vision** library)
   - OpenCV is a C++ computer vision library with Python cv2 bindings
   - We subscribe to the camera topic and convert images to proper `cv::Mat` or `NumPy array` data (it expects BGR encoding).
   - It provides a variety of image processing functions, and special ones for ArUco Markers identification, detection, and pose estimation.
3. **Alternative ROS2 camera drivers**
   - `usb_cam`: USB camera driver, widely used in ROS1
   - `opencv_cam`: Uses OpenCV backend for image capture

Camera Exercise ▪ ArUco Detection & Pose Estimation - Part 1: `Image` msg & `TransformStamped` msg

```
# This message contains an uncompressed image
# (0, 0) is at top-left corner of image

std_msgs/Header header # Header timestamp should be acquisition time of image
                            # Header frame_id should be optical frame of camera
                            # origin of frame should be optical center of cameara
                            # +x should point to the right in the image
                            # +y should point down in the image
                            # +z should point into to plane of the image
                            # If the frame_id here and the frame_id of the CameraInfo
                            # message associated with the image conflict
                            # the behavior is undefined

uint32 height               # image height, that is, number of rows
uint32 width                # image width, that is, number of columns

# The legal values for encoding are in file src/image_encodings.cpp
# If you want to standardize a new string format, join
# ros-users@lists.ros.org and send an email proposing a new encoding.

string encoding       # Encoding of pixels -- channel meaning, ordering, size
                      # taken from the list of strings in include/sensor_msgs/image_encodings.hpp

uint8 is_bigendian    # is this data bigendian?
uint32 step           # Full row length in bytes
uint8[] data          # actual matrix data, size is (step * rows)
```

ros2 interface show sensor_msgs/msg/Image

Camera Exercise ▪ ArUco Detection & Pose Estimation - Part 1: `Image` msg & `TransformStamped` msg

```
# This expresses a transform from coordinate frame header.frame_id
# to the coordinate frame child_frame_id at the time of header.stamp
#
# This message is mostly used by the
# <a href="https://index.ros.org/p/tf2/">tf2</a> package.
# See its documentation for more information.
#
# The child_frame_id is necessary in addition to the frame_id
# in the Header to communicate the full reference for the transform
# in a self contained message.

# The frame id in the header is used as the reference frame of this transform.
std_msgs/Header header

# The frame id of the child frame to which this transform points.
string child_frame_id

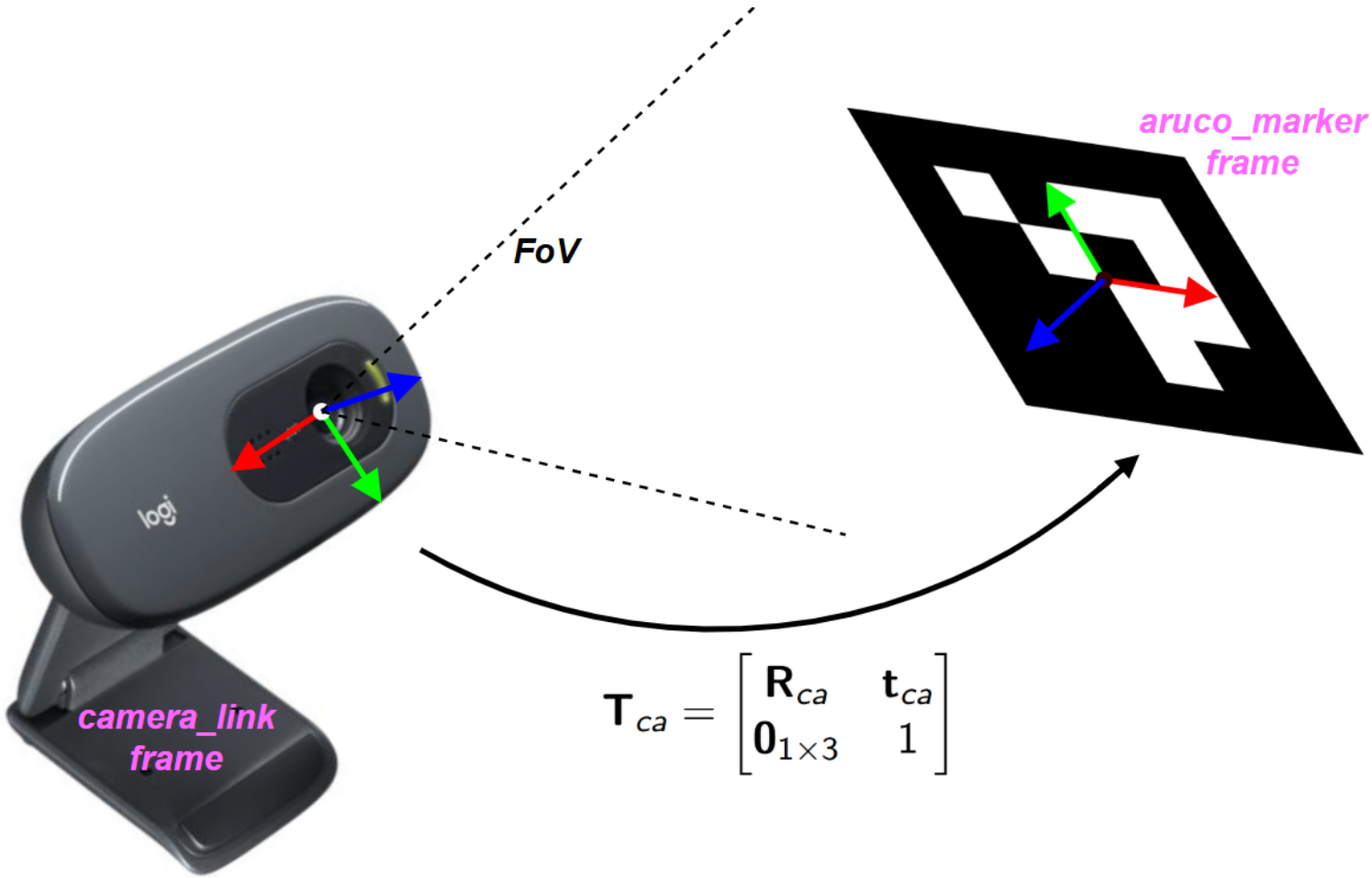
# Translation and rotation in 3-dimensions of child_frame_id from header.frame_id.
Transform transform
```

```
# This represents the transform between two coordinate frames in free space.

Vector3 translation
Quaternion rotation
```

ros2 interface show geometry_msgs/msg/TransformStamped & ros2 interface show geometry_msgs/msg/Transform

Camera Exercise ▪ ArUco Detection & Pose Estimation - Part 2: ArUco Setup

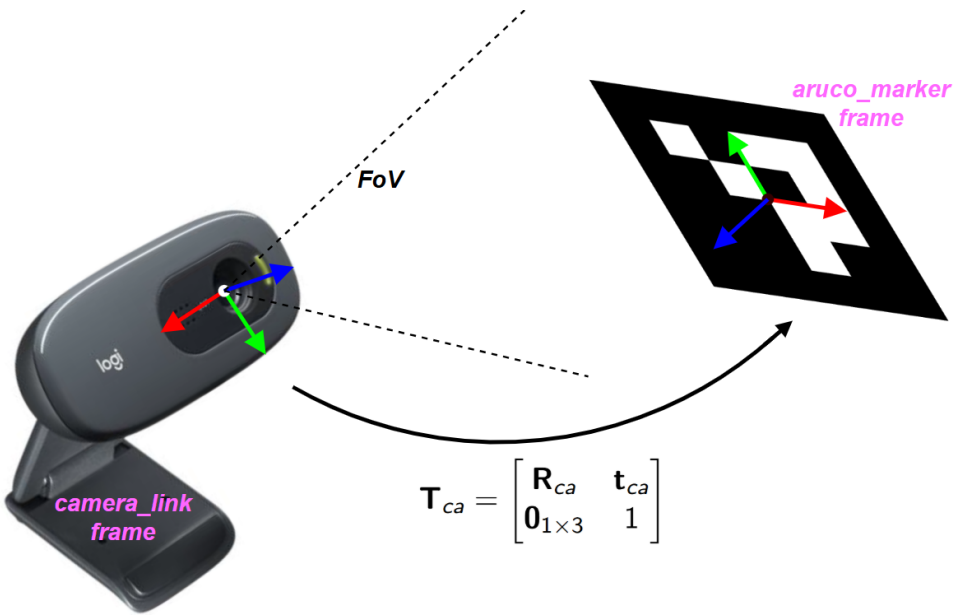


ArUco Detection and Pose Estimation Diagram

Camera Exercise ▪ ArUco Detection & Pose Estimation - Part 2: ArUco Setup

The `aruco_detection` node processes camera images to detect ArUco markers, estimate their 3D pose w.r.t camera, and publish both TF and visualization data for RViz2.

1. **Subscriber:** Listens to topic `/image_raw` (msg_type: `sensor_msgs/Image`) to receive raw camera frames. Each frame is converted with `CvBridge` and processed to detect an ArUco marker and compute its pose using the camera intrinsics.
2. **Publishers:** Publish a cube marker representing the detected ArUco on topic `/marker` (msg_type: `visualization_msgs/Marker`), and send the `camera_link` → `aruco_marker` transform (msg_type: `geometry_msgs/TransformStamped`) via a `TransformBroadcaster` for integration into the TF tree.

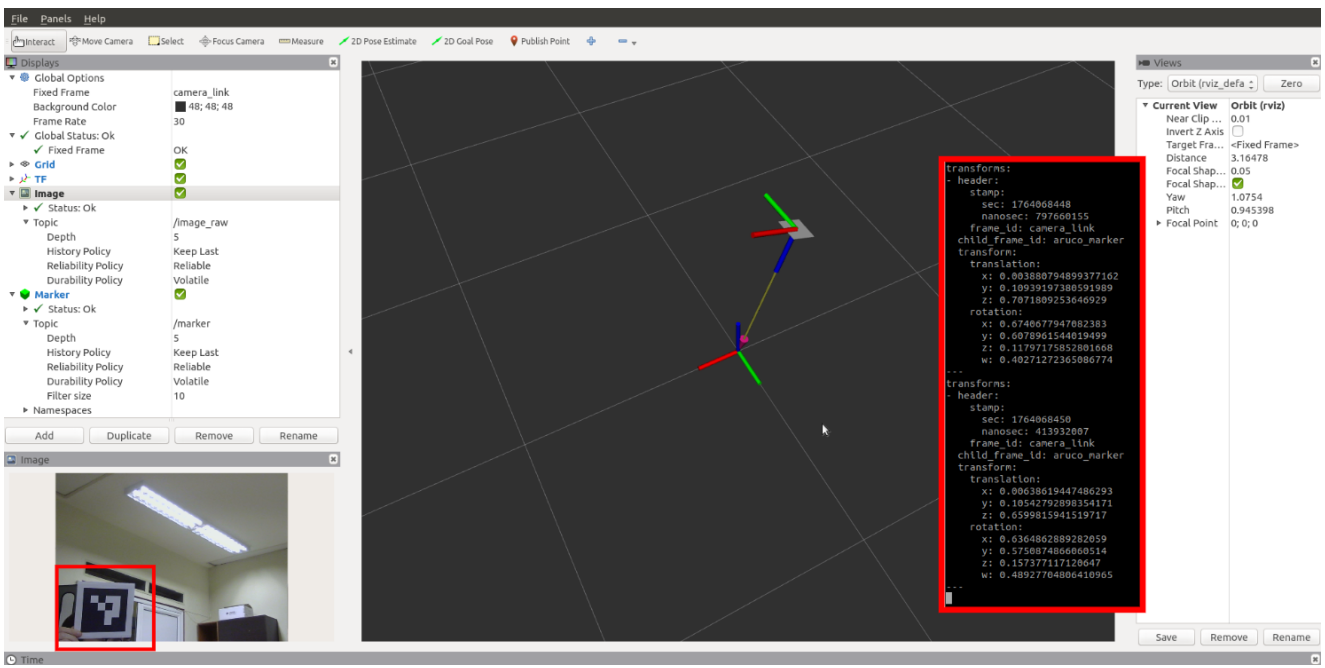


ArUco Detection and Pose Estimation Diagram

(it's a good practice to create some kind of filter, for example an exponential moving average filter, for smooth position and orientation estimations)

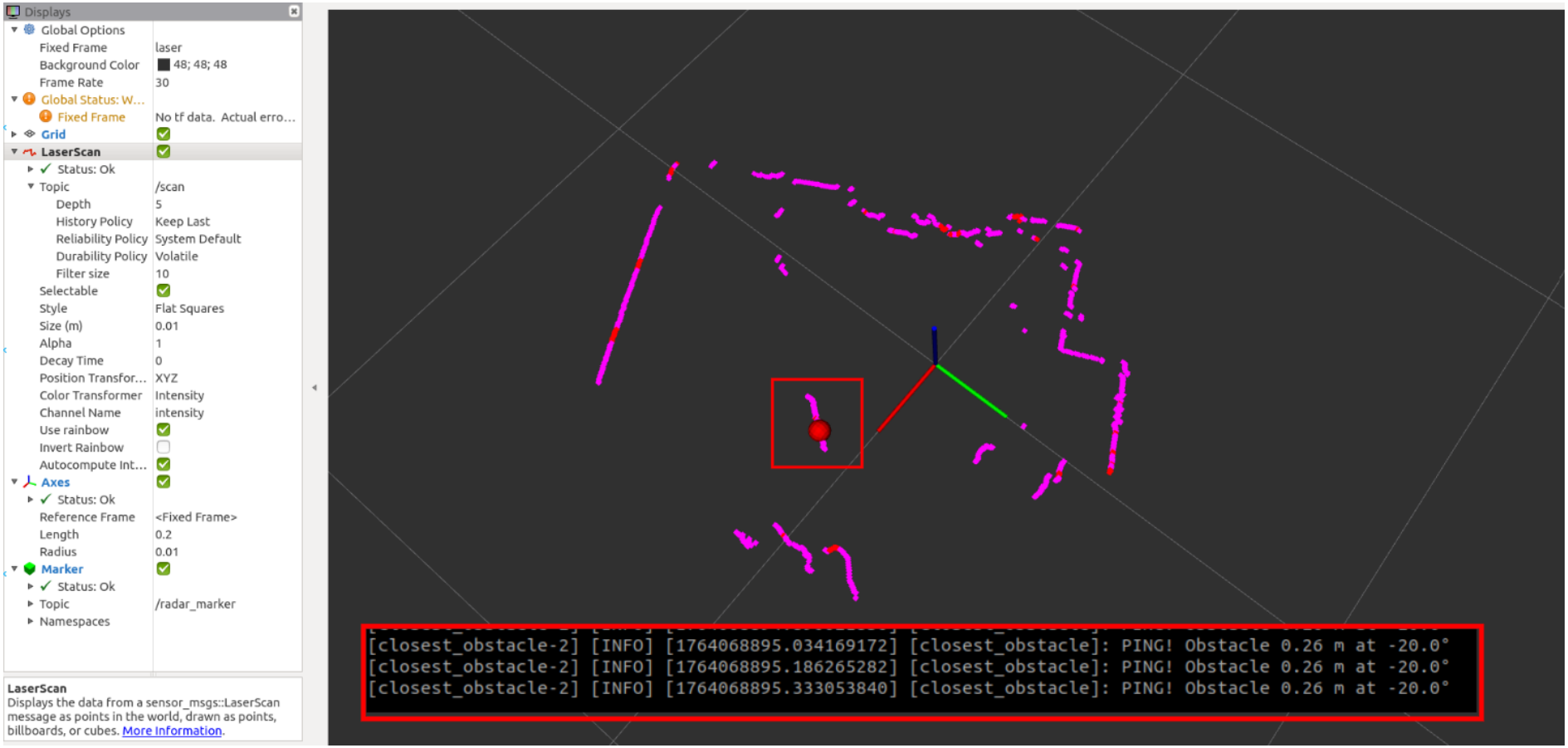
Camera Exercise ▪ ArUco Detection & Pose Estimation - Part 3: RViz2 Visualize

To visualize the detected ArUco marker, we use `visualization_msgs/msg/Marker` messages, in order to create a rectangular paralleliped with the Aruco marker's dimensions. We create a representation of type CUBE (we could have also used a more detailed mesh), and publish our marker to the topic `/marker`. To see it in RViz2, we need to add a **Marker Display**, that subscribes to the topic `/marker`. We also add an **Image Display**, that subscribes to the topic `/image_raw` and gives us the camera feed.



ArUco Detection and Pose Estimation - RViz2 view, the transformations are taken from `ros2 topic echo /tf`

LIDAR Exercise ▪ Distance Measurement & Radar Setup

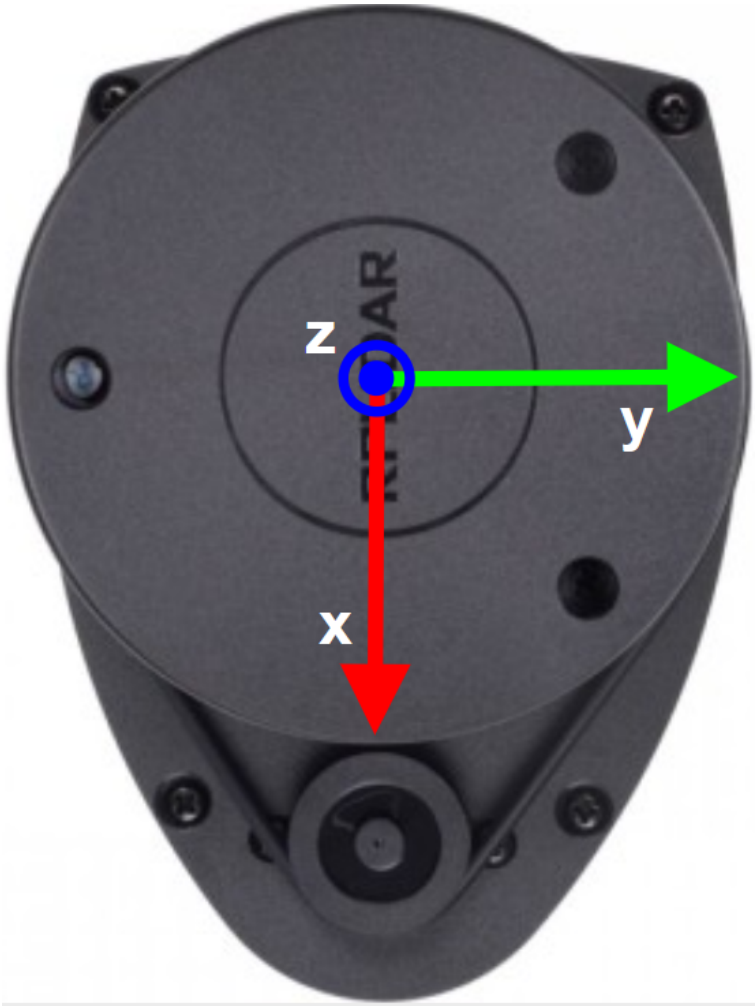


LIDAR and Radar setup - RViz2 view

LIDAR Exercise ▪ Distance Measurement & Radar Setup - RPLIDAR A1M8

We're going to use the **Slamtec RPILDAR A1M8**. Some of its specifics are:

1. **0.15–12 m detection range**
2. $\leq 1\%$ **to** $\leq 2\%$ **distance resolution** (depending on distance range, best for ≤ 12 m, worse for 12-16 m)
3. 360° **angular range** (scanning coverage)
4. $\leq 1^\circ$ **angular resolution**
5. **5–10 Hz scan rate** (rotational speed, typically 5.5 Hz, in a clockwise manner)
6. **Up to 8000 Hz sampling/data rate** (0.125 ms sample duration)
7. **785 nm laser wavelength** (IR/infrared)



RPLIDAR A1M8, along with its coordinate frame

LIDAR Exercise ▪ Distance Measurement & Radar Setup - rplidar_ros

We are going to use the ROS2 package **rplidar_ros** to interface with Slamtec RPLIDAR sensors and publish 2D LIDAR scan data into the ROS2 ecosystem.

1. **rplidar_ros**
   - ROS2 driver for RPLIDAR models (A1, A2, A3, etc.)
   - Publishes 2D scan data as `sensor_msgs/LaserScan` messages
   - Configurable parameters, such as scan frequency, angular resolution, frame ID and topic names
2. **Usage in ROS2**
   - Launch the driver using a provided or custom launch file, e.g. `rplidar.launch.py`
   - Subscribe to the `/scan` topic to receive real-time LIDAR ranges
   - Works with **SLAM packages**, **navigation stacks**, and **obstacle detection pipelines**

LIDAR Exercise ▪ Distance Measurement & Radar Setup - Part 1: LaserScan msg

```
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

std_msgs/Header header # timestamp in the header is the acquisition time of
                      # the first ray in the scan.
                      #
                      # in frame frame_id, angles are measured around
                      # the positive Z axis (counterclockwise, if Z is up)
                      # with zero angle being forward along the x axis

float32 angle_min     # start angle of the scan [rad]
float32 angle_max     # end angle of the scan [rad]
float32 angle_increment # angular distance between measurements [rad]

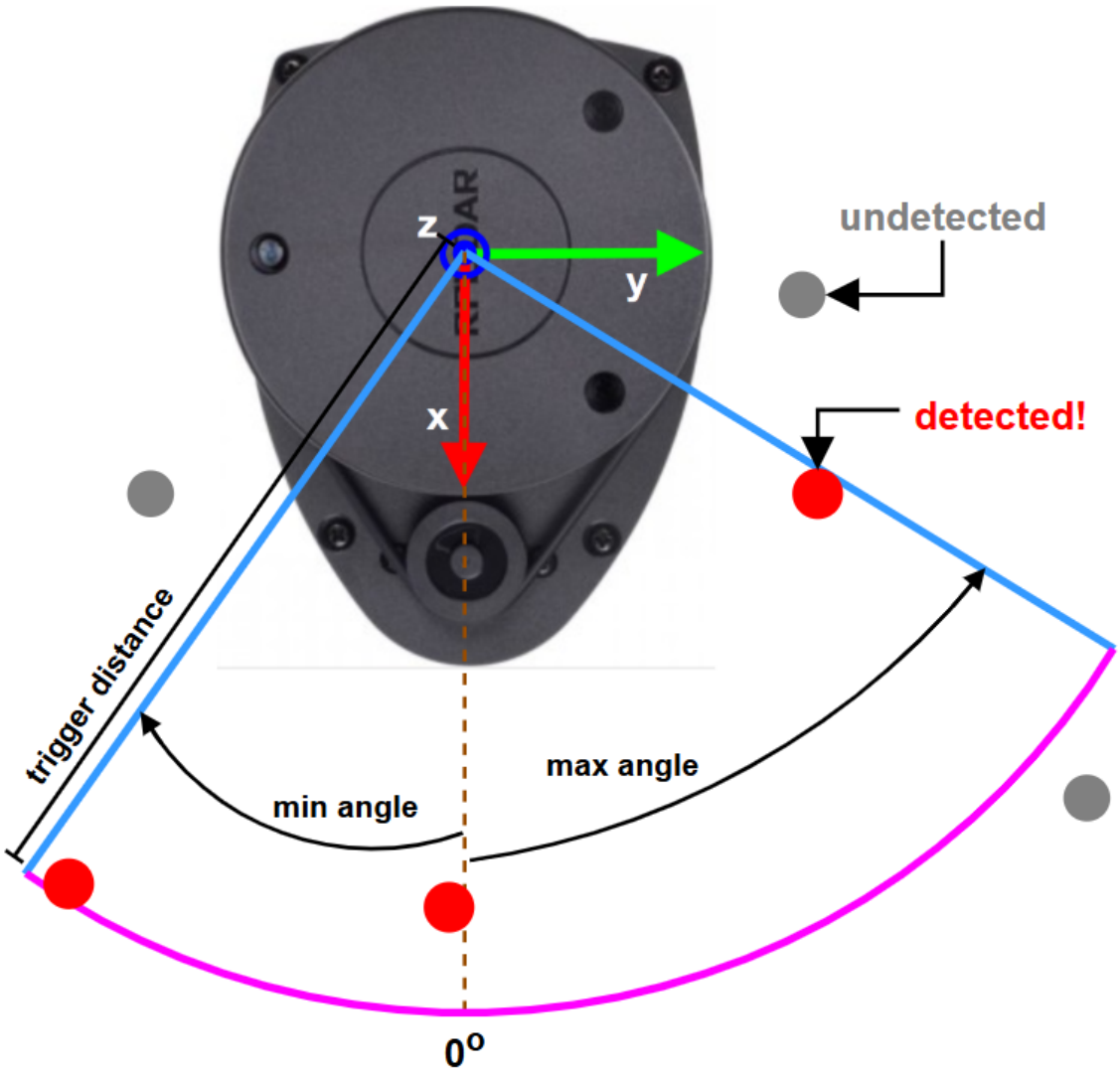
float32 time_increment # time between measurements [seconds] - if your scanner
                      # is moving, this will be used in interpolating position
                      # of 3d points
float32 scan_time     # time between scans [seconds]

float32 range_min     # minimum range value [m]
float32 range_max     # maximum range value [m]

float32[] ranges      # range data [m]
                      # (Note: values < range_min or > range_max should be discarded)
float32[] intensities # intensity data [device-specific units].  If your
                      # device does not provide intensities, please leave
                      # the array empty.
```

ros2 interface show sensor_msgs/msg/LaserScan

LIDAR Exercise ▪ Distance Measurement & Radar Setup - Part 2: Radar Setup



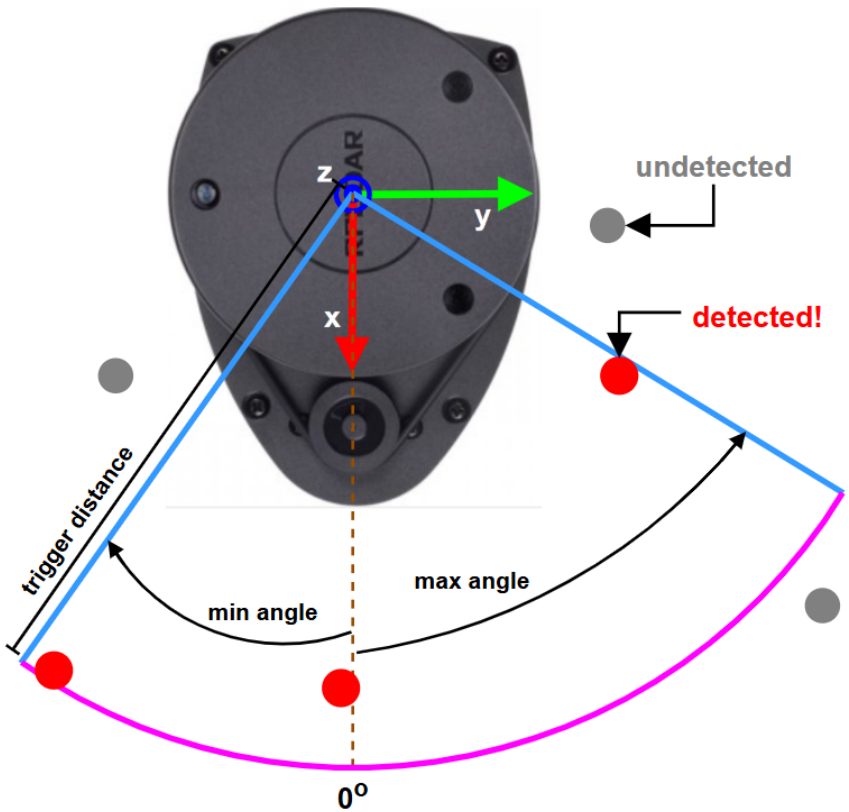
LIDAR - Radar Setup Diagram

It's worth noting that the LIDAR rotates clockwise when viewed from above. But, when the measurements are stored in a LaserScan message, the data is reordered and the start and end angles are remapped. This ensures that the scan angles increase counter-clockwise around the $+Z$ axis, following the right-hand rule convention used in ROS, as shown in the picture.

LIDAR Exercise ▪ Distance Measurement & Radar Setup - Part 2: Radar Setup

The node `closest_obstacle` monitors the surroundings using a 2D LiDAR and detects obstacles within a configurable angular window. It highlights detected obstacles in RViz2 using visualization markers.

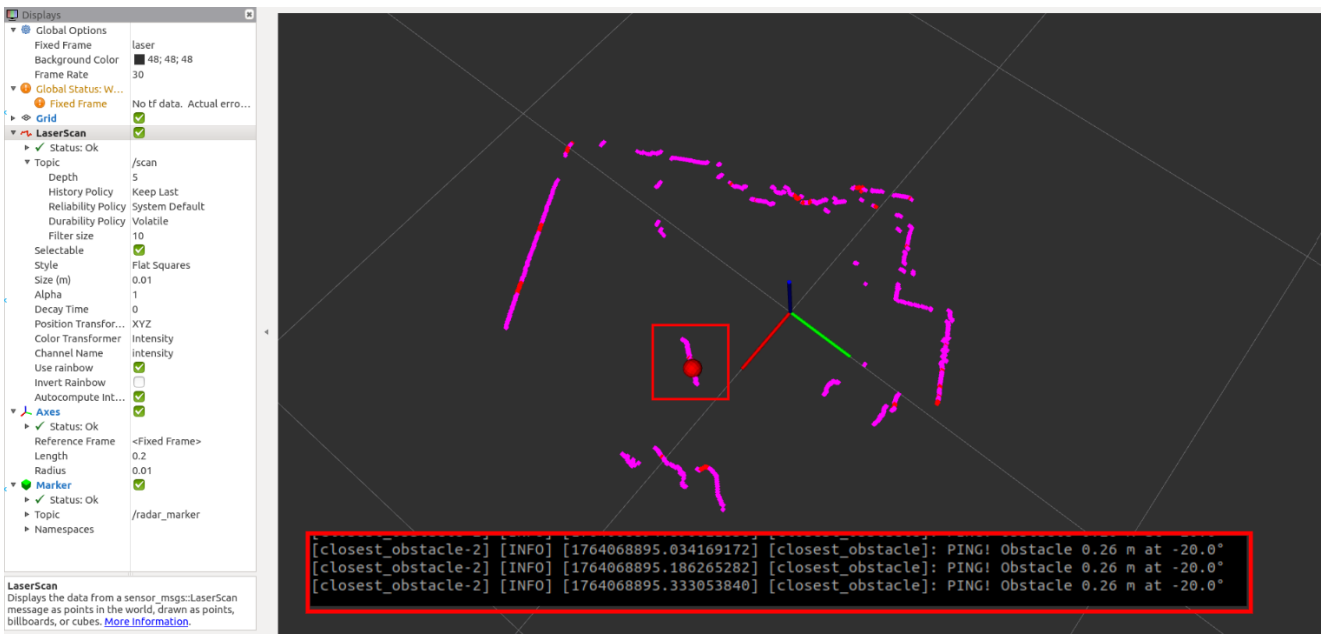
1. **Subscriber:** Listens to topic `/scan` (message type: `sensor_msgs/LaserScan`) to continuously receive LiDAR distance measurements. The node filters the scan to its circular sector and detects the closest obstacle within that region.
2. **Publisher:** Publishes a red sphere marker on `/radar_marker` (message type: `visualization_msgs/Marker`) to show the obstacle's location in RViz2. If no obstacle is detected or it is too far, the marker is removed.



LIDAR - Radar Setup Diagram

LIDAR Exercise ▪ Distance Measurement & Radar Setup - Part 3: RViz2 Visualize

In this exercise we also use the `visualization_msgs/msg/Marker` message, in order to publish (to the topic `/radar_marker`) a red sphere where an obstacle is detected inside the defined ***circular sector region*** on the xy plane (only the ***first detected obstacle is visualized***). So, we need to add a **Marker Display** in RViz2, that subscribes to the topic `/radar_marker`. We also need to add a **LaserScan Display**, that subscribes to the topic `/scan` to get the LIDAR data. We have effectively implemented a "sensor-level tripwire"!



LIDAR and radar setup - RViz2 view, the detected obstacle's distance and angle are also shown when running our node

References

- **Double Pendulum MAB ME-series Encoders**
- **Elephant Robotics MyArm 300 Pi Specifications and MyCobot 280 Pi Docs (including Encoders info)**
- **Intrinsic and Extrinsic Camera Parameters in Computer Vision**
- **Camera calibration process with OpenCV**
- **Logitech C270 HD WEBCAM and specs**
- **LIDAR Overview & How does it work? here_1 & here_2**
- **Slamtec RPLIDAR A1M8 and specs**
- **v4l2_camera ROS2 package**
- **OpenCV library for python (opencv-python)**
- **rplidar_ros ROS2 package**